

Dataset C – Inventory & Procurement Management System

Standard Response Structure

Assessment Objective

This assessment evaluates your ability to build a complete MERN Full Stack application involving:

- Authentication & Authorization
- External API Integration
- Data Validation & Sanitization
- MongoDB Persistence
- Relationship Modeling
- REST API Development
- Querying & Aggregation
- Frontend Integration
- Protected Routes
- State Management
- Deployment Readiness

The assessment focuses on engineering workflows rather than isolated CRUD operations.

Problem Statement

You are required to develop an:

Inventory & Procurement Management System

The system manages:

- Users
- Suppliers
- Products
- Purchase Orders
- Inventory Transactions

Students must:

1. Fetch inventory dataset from external API
2. Validate and sanitize records
3. Persist valid records into MongoDB Atlas
4. Build secure REST APIs
5. Implement role-based authorization
6. Build React frontend consuming backend APIs
7. Implement analytics & dashboard features

Success Response

```
{
  "success": true,
  "message": "Operation successful",
  "data": []
}
```

Error Response

```
{
  "success": false,
  "message": "Error message"
}
```

Section A — Authentication & Authorization

Objective

Implement JWT-based authentication and role-based access control.

Q1 — Register API

POST /auth/register

Requirements

- Register new user
- Password hashing mandatory
- Duplicate email prevention
- Store role

Roles

- admin
- procurement_manager
- warehouse_staff
- supplier

Q2 — Login API

POST /auth/login

Requirements

- JWT token generation
- Invalid credential handling
- Return authenticated user details

Q3 — Current User API

GET /auth/me

Requirements

- Protected route

- Return logged-in user details

Authorization Rules

Only:

- admin
- procurement_manager

can:

- create purchase orders
- approve procurement requests
- access analytics APIs
- manage suppliers

Warehouse staff can:

- update inventory
- manage stock transactions

Suppliers can:

- view assigned purchase orders
- update delivery status

Section B — Dataset Synchronization & MongoDB

Persistence

Objective

Fetch dataset from external API, validate records, sanitize values, and persist valid records into MongoDB Atlas.

Q4 — Sync API

POST /sync

Responsibilities

- Fetch dataset from private API
- Validate records
- Normalize values
- Reject invalid entries
- Prevent duplicate insertion
- Persist valid records into MongoDB
- Return sync summary

Example Response

```
{
  "success": true,
  "totalFetched": 120,
  "inserted": 95,
  "duplicates": 8,
  "rejected": 17
}
```

Required Sanitization Examples

Raw Value	Expected Value
" electronics "	"Electronics"
"ACTIVE"	"active"
" Dell Monitor "	"Dell Monitor"

Validation Requirements

Reject:

- invalid stock quantity
- invalid supplier references
- invalid dates
- duplicate product/order IDs
- invalid procurement status
- negative pricing values

Q5 — Health API

GET /health

Example

```
{
  "success": true,
  "database": "connected",
  "documentCount": 95
}
```

Section C — MongoDB Persistence, Relationships & Runtime Validation

Objective

Implement MongoDB persistence with proper relationships, validation logic, and runtime data consistency.

Required Collections & Relationships

User

Fields

- userId
- name
- email
- role
- department
- status

Supplier

Fields

- supplierId
- name
- email
- phone
- category
- status

Product

Relationships

- supplier → Supplier

Fields

- productId
- name
- category
- stockQuantity
- reorderLevel
- unitPrice
- status

Purchase Order

Relationships

- supplier → Supplier
- createdBy → User

Fields

- orderId
- products
- totalAmount
- orderDate
- deliveryDate
- status

Inventory Transaction

Relationships

- product → Product
- updatedBy → User

Fields

- transactionId
- type
- quantity
- previousStock
- updatedStock
- createdAt

Section D — CRUD & Workflow APIs

Objective

Implement REST APIs with proper validation and workflow handling.

Q6 — User APIs

GET /users
GET /users/:id

Q7 — Supplier APIs

POST /suppliers
GET /suppliers
GET /suppliers/:id
PATCH /suppliers/:id
DELETE /suppliers/:id

Q8 — Product APIs

POST /products
GET /products
GET /products/:id
PATCH /products/:id
DELETE /products/:id

Q9 — Purchase Order APIs

POST /orders
GET /orders
GET /orders/:id
PATCH /orders/:id
DELETE /orders/:id

Workflow Rules

- Orders cannot be created for inactive suppliers
- Negative stock quantities must be rejected
- Duplicate product names within same category not allowed
- Delivered orders cannot be modified
- Stock quantity must update after approved procurement

Section E — Procurement Workflow APIs

Objective

Implement protected workflow APIs.

Q10 — Approve Purchase Order

PATCH /orders/:id/approve

Requirements

- Only admin/procurement_manager allowed
- Order must exist
- Approved orders cannot be reapproved

Q11 — Update Delivery Status

PATCH /orders/:id/status

Allowed Status

- pending
- approved
- shipped
- delivered
- cancelled

Workflow Rules

- Cancelled orders cannot move to delivered
- Delivered orders automatically update inventory
- Suppliers cannot approve orders
- Warehouse staff cannot cancel approved orders

Section F — Filtering, Search & Pagination

Objective

Implement scalable querying APIs.

Q12 — Product Filters

Examples

GET /products?category=Electronics
GET /products?stockMin=10
GET /products?status=active

Q13 — Supplier Filters

Examples

GET /suppliers?status=active
GET /suppliers?category=Hardware

Q14 — Pagination & Search

Examples

GET /orders?page=1&limit=10
GET /products?search=monitor

Combined Query Example

GET /products?
category=Electronics&
status=active&
page=2&
limit=5

Section G — Analytics & Aggregation APIs

Objective

Implement MongoDB aggregation APIs.

Q15 — Inventory Analytics

GET /analytics/inventory

Return

- total products
- low stock products
- out-of-stock count
- active suppliers count

Q16 — Procurement Analytics

GET /analytics/procurement

Return

- total purchase orders
- delivered orders
- cancelled orders
- monthly procurement cost

Q17 — Supplier Analytics

GET /analytics/suppliers

Return

- supplier-wise order count
- highest procurement value
- delayed deliveries
- active supplier percentage

Section H — Frontend Integration

Objective

Build React frontend consuming backend APIs.

Mandatory Frontend Features

Authentication

- Login page
- JWT token handling
- Protected routes
- Role-based rendering

Dashboard

Display:

- inventory analytics cards
- low stock alerts
- recent purchase orders
- supplier summaries

Warehouse Module

- update stock
- view inventory
- filter products
- track stock changes

Procurement Manager Module

- manage suppliers
- approve orders
- view procurement analytics
- track deliveries

Mandatory State Management

Use:

- Context API
- Reducer

Section I — Frontend Evaluator Compliance, Global State & Protected Routes (5 Marks)

Objective

Evaluate frontend architecture.

Requirements

Implement:

- Protected routes
- Role-based navigation
- Reducer logic
- Global state management
- Required data-testid attributes
- Dynamic UI updates based on global state

Mandatory Global State

Your application must maintain global state for:

- authenticated user
- JWT token
- suppliers
- products
- orders
- inventoryTransactions
- filters/search values
- analytics/dashboard data

Use:

- Context API
- Reducer

Mandatory data-testid Attributes

Authentication

Component	data-testid
Login form	login-form
Email input	email-input
Password input	password-input
Login button	login-btn
Logout button	logout-btn

Navigation

Component	data-testid
Navbar	navbar
Dashboard link	dashboard-link
Suppliers link	suppliers-link
Products link	products-link
Orders link	orders-link
Inventory link	inventory-link

Dashboard

Component	data-testid
Analytics container	analytics-container
Total products card	total-products-card
Low stock card	low-stock-card
Active suppliers card	active-suppliers-card
Procurement chart	procurement-chart
Recent orders section	recent-orders

Product Module

Component	data-testid
Product table	product-table
Product search input	product-search
Product filter dropdown	product-filter
Product row	product-row

Order Module

Component	data-testid
Order list	order-list
Create order button	create-order-btn
Approve order button	approve-order-btn
Order search input	order-search
Pagination next button	pagination-next
Pagination previous button	pagination-prev

Inventory Module

Component	data-testid
Inventory table	inventory-table
Update stock button	update-stock-btn
Inventory row	inventory-row

Section J — Frontend Evaluator Compliance & Deployment (10 Marks)

Objective

Validate deployment readiness, frontend evaluator compatibility, and runtime UI behavior using automated Playwright-based evaluation.

Mandatory Frontend Behavior

Your frontend must:

- consume your own backend APIs
- update UI dynamically after CRUD operations
- maintain JWT token in global state
- maintain authenticated session after refresh
- restrict unauthorized route access
- render UI based on user role

Mandatory React Router Routes

Students must implement the following routes using React Router DOM.

Route	Purpose
/login	Authentication
/dashboard	Analytics dashboard
/suppliers	Supplier management
/products	Product management
/orders	Purchase order management
/inventory	Inventory tracking
/profile	Current user profile

Global State Exposure Requirement

To support automated Playwright evaluation, students must expose global application state using: `window.appState`

The global state object must dynamically reflect the current application state.

Mandatory window.appState Structure

```
window.appState = {
  authUser,
  token,
  suppliers,
  products,
  orders,
  inventoryTransactions,
  filters,
  analytics }
```